



Verifiable Delay Functions

Dan Boneh¹, Joseph Bonneau², Benedikt Bünz¹, and Ben Fisch¹(✉)

¹ Stanford University, Stanford, USA
{dabo,bfisch}@cs.stanford.edu

² New York University, New York, USA

Abstract. We study the problem of building a *verifiable delay function* (VDF). A VDF requires a specified number of sequential steps to evaluate, yet produces a unique output that can be efficiently and publicly verified. VDFs have many applications in decentralized systems, including public randomness beacons, leader election in consensus protocols, and proofs of replication. We formalize the requirements for VDFs and present new candidate constructions that are the first to achieve an exponential gap between evaluation and verification time.

1 Introduction

Consider the problem of running a verifiable lottery using a *randomness beacon*, a concept first described by Rabin [62] as an ideal service that regularly publishes random values which no party can predict or manipulate. A classic approach is to apply an extractor function to a public entropy source, such as stock prices [24]. Stock prices are believed to be difficult to predict for a passive observer, but an active adversary could manipulate prices to bias the lottery. For example, a high-frequency trader might slightly alter the closing price of a stock by executing (or not executing) a few transactions immediately before the market closes.

Suppose the extractor takes only a single bit per asset (e.g. whether the stock finished up or down for the day) and suppose the adversary is capable of changing this bit for k different assets using last-second trades. The attacker could read the prices of the assets it cannot control, quickly simulate 2^k potential lottery outcomes based on different combinations of the k outcomes it can control, and then manipulate the market to ensure its preferred lottery outcome occurs.

One solution is to add a delay function after extraction, making it slow to compute the beacon outcome from an input of raw stock prices. With a delay function of say, one hour, by the time the adversary simulates the outcome of any potential manipulation strategy, the market will be closed and prices finalized, making it too late to launch an attack. This suggests the key security property for a delay function: it should be infeasible for an adversary to distinguish the function's output from random in less than a specified amount of wall-clock time, even given a potentially large number of parallel processors.

A trivial delay function can be built by iterating a cryptographic hash function. For example, it is reasonable to assume it is infeasible to compute 2^{40}

iterations of SHA-256 in a matter of seconds, even using specialized hardware. However, a lottery participant wishing to verify the output of this delay function must repeat the computation in its entirety (which might take many hours on a personal computer). Ideally, we would like to design a delay function which any observer can quickly verify was computed correctly.

Defining delay functions. In this paper we formalize the requirements for a *verifiable delay function* (VDF) and provide the first constructions which meet these requirements. A VDF consists of a triple of algorithms: **Setup**, **Eval**, and **Verify**. **Setup**(λ, t) takes a security parameter λ and delay parameter t and outputs public parameters **pp** (which fix the domain and range of the VDF and may include other information necessary to compute or verify it). **Eval**(**pp**, x) takes an input x from the domain and outputs a value y in the range and (optionally) a short proof π . Finally, **Verify**(**pp**, x, y, π) efficiently verifies that y is the correct output on x . Crucially, for every input x there should be a *unique* output y that will verify. Informally, a VDF scheme should satisfy the following properties:

- *sequential*: honest parties can compute $(y, \pi) \leftarrow \text{Eval}(\mathbf{pp}, x)$ in t sequential steps, while no parallel-machine adversary with a polynomial number of processors can distinguish the output y from random in significantly fewer steps.
- *efficiently verifiable*: We prefer **Verify** to be as fast as possible for honest parties to compute; we require it to take total time $O(\text{polylog}(t))$.

A VDF should remain secure even in the face of an attacker able to perform a polynomially bounded amount of pre-computation.

Some VDFs may also offer additional useful properties:

- *decodable*: An input x can be recovered uniquely from an output y . If the decoding is efficient then no additional proof is required. For example, an invertible function or permutation that is sequentially slow to compute but efficient to invert could be used to instantiate an efficiently decodable VDF.
- *incremental*: a single set of public parameters **pp** supports multiple hardness parameters t . The number of steps used to compute y is specified in the proof, instead of being fixed during **Setup**.

Classic slow functions. Time-lock puzzles [64] are similar to VDFs in that they involve computing an inherently sequential function. An elegant solution uses repeated squaring in an RSA group as a time-lock puzzle. However, time-lock puzzles are not required to be universally verifiable and in all known constructions the verifier uses its secret state to prepare each puzzle and verify the results. VDFs, by contrast, may require an initial trusted setup but then must be usable on any randomly chosen input.

Another construction for a slow function dating to Dwork and Naor [31] is extracting modular square roots. Given a challenge $x \in \mathbb{Z}_p^*$ (with $p \equiv 3 \pmod{4}$), computing $y = \sqrt{x} = x^{\frac{p+1}{4}} \pmod{p}$ can be efficiently verified by checking that $y^2 = x \pmod{p}$. There is no known algorithm for computing modular exponentiation which is sublinear in the bit-length of the exponent. However, the

difficulty of puzzles is limited to $t = O(\log p)$ as the exponent can be reduced modulo $p - 1$ before computation, requiring the use of a very large prime p to produce a difficult puzzle. While it was not originally proposed for its sequential nature, it has subsequently been considered as such several times [39, 46]. In particular, Lenstra and Wesolowski [46] proposed chaining a series of such puzzles together in a construction called Sloth, with lotteries as a specific motivation. Sloth is best characterized as a *time-asymmetric encoding*, offering a trade-off in practice between computation and inversion (verification), and thus can be viewed as a pseudo-VDF. However, it does not meet our asymptotic definition of a VDF because it does not offer asymptotically efficient verification: the t -bit modular exponentiation can be computed in parallel time t , whereas the output (a t -bit number) requires $\Omega(t)$ time simply to read, and therefore verification cannot run in total time $\text{polylog}(t)$. We give a more complete overview of related work in Sect. 8.

Our contributions: In addition to providing the first formal definitions of VDFs, we contribute the following candidate constructions and techniques:

1. A theoretical VDF can be constructed using *incrementally verifiable computation* [66] (IVC), in which a proof of correctness for a computation of length t can be computed in parallel to the computation with only $\text{polylog}(t)$ processors. We prove security of this theoretical VDF using IVC as a black box. IVC can be constructed from succinct non-interactive arguments of knowledge (SNARKs) under a suitable extractor complexity assumption [14].
2. We propose a construction based on injective polynomials over algebraic sets that cannot be inverted faster than computing polynomial GCDs. Computing polynomial GCD is sequential in the degree d of the polynomials on machines with fewer than $O(d^2)$ processors. We propose a candidate construction of time-asymmetric encodings from a particular family of *permutation polynomials* over finite fields [37]. This construction is asymptotically a strict improvement on Sloth, and to the best of our knowledge is the first encoding offering an exponential time gap between evaluation and inversion. We call this a decodable *weak VDF* because it requires the honest **Eval** to use greater than $\text{polylog}(t)$ parallelism to run in parallel time t (the delay parameter).
3. We describe a practical boost to constructing VDFs from IVC using time-asymmetric encodings as the underlying sequential computation, offering up to a 20,000 fold improvement (in the SNARK efficiency) over naive hash chains. In this construction decodability of the VDF is maintained, however a SNARK proof is used to boost the efficiency of verification.
4. We construct a VDF secure against bounded pre-computation attacks following a generalization of time-lock puzzles based on exponentiation in a group of unknown order.

2 Applications

Before giving precise definitions and describing our constructions, we first informally sketch several important applications of VDFs in decentralized systems.

Randomness beacons. VDFs are useful for constructing randomness beacons from sources such as stock prices [24] or proof-of-work blockchains (e.g. Bitcoin, Ethereum) [12, 17, 60]. Proof-of-work blockchains include randomly sampled solutions to computational puzzles that network participants (called *miners*) continually find and publish for monetary rewards. Underpinning the security of proof-of-work blockchains is the strong belief that these solutions have high computational min-entropy. However, similar to potential manipulation of asset prices by high-frequency traders, powerful miners could potentially manipulate the beacon result by refusing to post blocks which produce an unfavorable beacon output.

Again, this attack is only feasible if the beacon can be computed quickly, as each block is fixed to a specific predecessor and will become “stale” if not published. If a VDF with a suitably long delay is used to compute the beacon, miners will not be able to determine the beacon output from a given block before it becomes stale. More specifically, given the desired delay parameter t , the public parameters $\mathbf{pp} = (\mathbf{ek}, \mathbf{vk}) \xleftarrow{\mathcal{R}} \text{Setup}(\lambda, t)$ are posted on the blockchain, then given a block b the beacon value is determined to be r where $(r, \pi) = \text{Eval}(\mathbf{ek}, b)$, and anyone can verify correctness by running $\text{Verify}(\mathbf{vk}, b, r, \pi)$. The security of this construction, and in particular the length of delay parameter which would be sufficient to prevent attacks, remains an informal conjecture due to the lack of a complete game-theoretic model capturing miner incentives in Nakamoto-style consensus protocols. We refer the reader to [12, 17, 60] for proposed models for blockchain manipulation. Note that most formal models for Nakamoto-style consensus such as that of Garay et al. [34] do not capture miners with external incentives such as profiting from lottery manipulation.

Another approach for constructing beacons derives randomness from a collection of participants, such as all participants in a lottery [36, 46]. The simplest paradigm is “commit-and-reveal” paradigm where n parties submit commitments to random values $r_1, \dots, r_n$ in an initial phase and subsequently reveal their commitments, at which point the beacon output is computed as $r = \bigoplus_i r_i$. The problem with this approach is that a malicious adversary (possibly controlling a number of parties) might manipulate the outcome by refusing to open its commitment after seeing the other revealed values, forcing a protocol restart. Lenstra and Wesolowski proposed a solution to this problem (called “Unicorn” [46]) using a delay function: instead of using commitments, each participant posts their r_i directly and $\text{seed} = H(r_1, \dots, r_n)$ is passed through a VDF. The outcome of Eval is then posted and can be efficiently verified. With a sufficiently long delay parameter (longer than the time period during which values may be submitted), even the last party to publish their r_i cannot predict what its impact will be on the final beacon outcome. The beacon is unpredictable even to an adversary who controls $n - 1$ of the participating parties. It has linear communication complexity and uses only two rounds. This stands in contrast to coin-tossing beacons which use verifiable secret sharing and are at best resistant to an adversary who controls a minority of the nodes [1, 23, 65]. These beacons also use super-linear communication and require multiple rounds of interaction. In the two party

setting there are tight bounds that an r -round coin-flipping protocol can be biased with $O(1/r)$ bias [54]. The “Unicorn” construction circumvents these bounds by assuming semi-synchronous communication, i.e. there exists a bound to how long an adversary can delay messages.

Resource-efficient blockchains. Amid growing concerns over the long-term sustainability of proof-of-work blockchains like Bitcoin, which consume a large (and growing) amount of energy, there has been concerted effort to develop resource-efficient blockchains in which miners invest an upfront capital expenditure which can then be re-used for mining. Examples include proof-of-stake [13, 28, 43, 44, 52], proof-of-space [58], and proof-of-storage [2, 53]. However, resource-efficient mining suffers from *costless simulation* attacks. Intuitively, since mining is not computationally expensive, miners can attempt to produce many separate forks easily.

One method to counter simulation attacks is to use a randomness beacon to select new leaders at regular intervals, with the probability of becoming a leader biased by the quality of proofs (i.e. amount of stake, space, etc.) submitted by miners. A number of existing blockchains already construct beacons from tools such as *verifiable random functions*, *verifiable secret sharing*, or *deterministic threshold signatures* [4, 23, 28, 43]. However, the security of these beacons requires a non-colluding honest majority; with a VDF-based lottery as described above this can potentially be improved to participation of any honest party.

A second approach, proposed by Cohen [26], is to combine proofs-of-resources with incremental VDFs and use the product of resources proved and delay induced as a measure of blockchain quality. This requires a proof-of-resource which is costly to initialize (such as certain types of proof-of-space). This is important such that the resources are committed to the blockchain and cannot be used for other purposes. A miner controlling N units of total resources can initialize a proof π demonstrating control over these N units. Further assume that the proof is non-malleable and that in each epoch there is a common random challenge c , e.g. a block found in the previous epoch, and let H be a random oracle available to everyone. In each epoch, the miner finds $\tau = \min_{1 \leq i \leq N} \{H(c, \pi, i)\}$ and computes a VDF on input c with a delay proportional to τ . The first miner to successfully compute the VDF can broadcast their block successfully. Note that this process mimics the random delay to find a Bitcoin block (weighted by the amount of resources controlled by each miner), but without each miner running a large parallel computation.

Proof of data replication. Another promising application of VDFs is proofs of replication, a special type of proof of storage of data which requires dedicating storage even if the data is publicly available. For instance, this could be used to prove that a number of replicas of the same file are being stored. Classic *proofs of retrievability* [41] are typically defined in a private-key client/server setting, where the server proves to the client that it can retrieve the client’s (private) data, which the client verifies using a private key.

Instead, the goal of a *proof of replication* [2, 3, 6] is to verify that a given server is storing a *unique replica* of some data which may be publicly

available. Armknecht et al. [6] proposed a protocol in the private verifier model using RSA time-lock puzzles. Given an efficiently decodable VDF, we can adapt their construction to create proofs-of-replication which are more transparent (i.e. do not rely on a designated verifier). Given a unique replicator identifier id and public parameters $\mathbf{pp} \xleftarrow{\mathcal{R}} \text{Setup}(\lambda, t)$, the replicator computes a unique *slow* encoding of the file that take sequential time t . This encoding is computed by breaking the file into b -bit blocks $B_1, \dots, B_n$ and storing $y_1, \dots, y_n$ where $(y_i, \perp) = \text{Eval}(\mathbf{pp}, B_i \oplus H(id||i))$ where H is a collision-resistant hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^b$. To verify that the replicator has stored this unique copy, a verifier can query an encoded block y_i (which must be returned in significantly less time than it is feasible to compute Eval). The verifier can quickly decode this response and check it for correctness, proving that the replicator has stored (or can quickly retrieve from somewhere) an encoding of this block which is unique to the identifier id . If the unique block encoding y_i has not been stored, the VDF ensures that it cannot be re-computed quickly enough to fool the verifier, even given access to B_i . The verifier can query for as many blocks as desired; each query has a $1 - \rho$ chance of exposing a cheating prover that is only storing a fraction ρ of the encoded blocks. Note that in this application it is critical that the VDF is decodable. Otherwise, the encoding of the file isn't a useful replica because it cannot be used to recover the data if all other copies are lost.

Computational timestamping. All known proof-of-stake systems are vulnerable to long-range forks due to post-hoc stakeholder misbehavior [13, 43, 44, 52]. In proof-of-stake protocols, at any given time the current stakeholders in the system are given voting power proportionate to their stake in the system. An honest majority (or supermajority) is assumed because the current stakeholders are incentivized to keep the system running correctly. However, after stakeholders have divested they no longer have this incentive. Once the majority (eq. supermajority) of stakeholders from a point in time in the past are divested, they can collude (or sell their key material to an attacker) in order to create a long alternate history of the system up until the present. Current protocols typically assume this is prevented through an external timestamping mechanism which can prove to users that the genuine history of the system is much older.

Incremental VDFs can provide computational evidence that a given version of the state's system is older (and therefore genuine) by proving that a long-running VDF computation has been performed on the genuine history just after the point of divergence with the fraudulent history. This potentially enables detecting long-range forks without relying on external timestamping mechanisms.

We note however that this application of VDFs is fragile as it requires precise bounds on the attacker's computation speed. For other applications (such as randomness beacons) it may be acceptable if the adversary can speed up VDF evaluation by a factor of 10 using faster hardware; a higher t can be chosen until even the adversary cannot manipulate the beacon even with a hardware speedup. For computational timestamping, a 10-fold speedup would be a serious problem: once the fraudulent history is more than one-tenth as old as the genuine history, an attacker can fool participants into believing the fraudulent history is actually *older* than the genuine one.

3 Model and Definitions

We now define VDFs more precisely. In what follows we say that an algorithm runs in *parallel time* t with p processors if it can be implemented on a PRAM machine with p parallel processors running in time t . We say *total time* (eq. sequential time) to refer to the time needed for computation on a single processor.

Definition 1. A VDF $V = (\text{Setup}, \text{Eval}, \text{Verify})$ is a triple of algorithms as follows:

- $\text{Setup}(\lambda, t) \rightarrow \mathbf{pp} = (\mathbf{ek}, \mathbf{vk})$ is a randomized algorithm that takes a security parameter λ and a desired puzzle difficulty t and produces public parameters $\mathbf{pp}$ that consists of an evaluation key $\mathbf{ek}$ and a verification key $\mathbf{vk}$. We require Setup to be polynomial-time in λ . By convention, the public parameters specify an input space $\mathcal{X}$ and an output space $\mathcal{Y}$. We assume that $\mathcal{X}$ is efficiently sampleable. Setup might need secret randomness, leading to a scheme requiring a trusted setup. For meaningful security, the puzzle difficulty t is restricted to be sub-exponentially sized in λ .
- $\text{Eval}(\mathbf{ek}, x) \rightarrow (y, \pi)$ takes an input $x \in \mathcal{X}$ and produces an output $y \in \mathcal{Y}$ and a (possibly empty) proof π . Eval may use random bits to generate the proof π but not to compute y . For all $\mathbf{pp}$ generated by $\text{Setup}(\lambda, t)$ and all $x \in \mathcal{X}$, algorithm $\text{Eval}(\mathbf{ek}, x)$ must run in parallel time t with $\text{poly}(\log(t), \lambda)$ processors.
- $\text{Verify}(\mathbf{vk}, x, y, \pi) \rightarrow \{\text{Yes}, \text{No}\}$ is a deterministic algorithm takes an input, output and proof and outputs Yes or No. Algorithm Verify must run in total time polynomial in $\log t$ and λ . Notice that Verify is much faster than Eval .

Additionally V must satisfy Correctness (Definition 2), Soundness (Definition 3), and Sequentiality (Definition 4).

Correctness and Soundness. Every output of Eval must be accepted by Verify . We guarantee that the output y for an input x is unique because Eval evaluates a deterministic function on $\mathcal{X}$. Note that we do not require the proof π to be unique, but we do require that the proof is sound and that a verifier cannot be convinced that some different output is the correct VDF outcome. More formally,

Definition 2 (Correctness). A VDF V is correct if for all λ, t , parameters $(\mathbf{ek}, \mathbf{vk}) \xleftarrow{\mathbf{R}} \text{Setup}(\lambda, t)$, and all $x \in \mathcal{X}$, if $(y, \pi) \xleftarrow{\mathbf{R}} \text{Eval}(\mathbf{ek}, x)$ then $\text{Verify}(\mathbf{vk}, x, y, \pi) = \text{Yes}$.

We also require that for no input x can an adversary get a verifier to accept an incorrect VDF output.

Definition 3 (Soundness). A VDF is sound if for all algorithms $\mathcal{A}$ that run in time $O(\text{poly}(t, \lambda))$

$$P \left[\text{Verify}(\mathbf{vk}, x, y, \pi) = \text{Yes} \mid \begin{array}{l} \mathbf{pp} = (\mathbf{ek}, \mathbf{vk}) \xleftarrow{\mathbf{R}} \text{Setup}(\lambda, t) \\ (x, y, \pi) \xleftarrow{\mathbf{R}} \mathcal{A}(\lambda, \mathbf{pp}, t) \end{array} \right] = \text{negl}(\lambda)$$

Size restriction on t . Asymptotically t must be subexponential in λ . The reason for this is that the adversary needs to be able to run in time at least t (Eval requires this), and if t is exponential in λ then the adversary might be able to break the underlying computational security assumptions that underpin both the soundness as well as the sequentiality of the VDF, which we will formalize next.

Parallelism in Eval. The practical implication of allowing more parallelism in Eval is that “honest” evaluators may be required to have this much parallelism in order to complete the challenge in time t . The sequentiality security argument will compare an adversary’s advantage to this optimal implementation of Eval. Constructions of VDFs that do not require any parallelism to evaluate Eval in the optimal number of sequential steps are obviously superior. However, it is unlikely that such constructions exist (without trusted hardware). Even computing an iterated hash function or modular exponentiation (used for time-lock puzzles) could be computed faster by parallelizing the hash function or modular arithmetic. In fact, for an decodable VDF it is necessary that $|\mathcal{Y}| > \text{poly}(t)$, and thus the challenge inputs to Eval have size $\text{poly} \log(t)$. Therefore, in our definition we allow algorithm Eval up to $\text{poly} \log(t)$ parallelism.

3.1 VDF Security

We call the security property needed for a VDF scheme σ -*sequentiality*. Essentially, we require that no adversary is able to compute an output for Eval on a random challenge in parallel time $\sigma(t) < t$, even with up to “many” parallel processors and after a potentially large amount of pre-computation. It is critical to bound the adversary’s allowed parallelism, and we incorporate this into the definition. Note that for an efficiently decodable VDF, an adversary with $|\mathcal{Y}|$ processors can always compute outputs in $o(t)$ parallel time by simultaneously trying all possible outputs in $\mathcal{Y}$. This means that for efficiently decodable VDFs it is necessary that $|\mathcal{Y}| > \text{poly}(t)$, and cannot achieve σ -sequentiality against an adversary with greater than $|\mathcal{Y}|$ processors.

We define the following sequentiality game applied to an adversary $\mathcal{A} := (\mathcal{A}_0, \mathcal{A}_1)$:

$\mathbf{pp} \xleftarrow{\mathcal{R}} \text{Setup}(\lambda, t)$	//	choose a random $\mathbf{pp}$
$L \xleftarrow{\mathcal{R}} \mathcal{A}_0(\lambda, \mathbf{pp}, t)$	//	adversary preprocesses $\mathbf{pp}$
$x \xleftarrow{\mathcal{R}} \mathcal{X}$	//	choose a random input x
$y_A \xleftarrow{\mathcal{R}} \mathcal{A}_1(L, \mathbf{pp}, x)$	//	adversary computes an output y_A

We say that $(\mathcal{A}_0, \mathcal{A}_1)$ wins the game if $y_A = y$ where $(y, \pi) := \text{Eval}(\mathbf{pp}, x)$.

Definition 4 (Sequentiality). For functions $\sigma(t)$ and $p(t)$, the VDF is (p, σ) -sequential if no pair of randomized algorithms $\mathcal{A}_0$, which runs in total time $O(\text{poly}(t, \lambda))$, and $\mathcal{A}_1$, which runs in parallel time $\sigma(t)$ on at most $p(t)$ processors, can win the sequentiality game with probability greater than $\text{negl}(\lambda)$.

The definition captures the fact that even after $\mathcal{A}_0$ computes on the parameters $\mathbf{pp}$ for a (polynomially) long time, the adversary $\mathcal{A}_1$ cannot compute an output from the input x in time $\sigma(t)$ on $p(t)$ parallel processors. If a VDF is (p, σ) -sequential for any polynomial p , then we simply say the VDF is σ -sequential. In the sequentiality game we do not require the online attack algorithm $\mathcal{A}_1$ to output a proof π . The reason is that in many of our applications, for example in a lottery, the adversary can profit simply by learning the output early, even without being able to prove correctness to a third party.

Values of $\sigma(t)$. Clearly any candidate construction trivially satisfies $\sigma(t)$ -sequentiality for *some* σ (e.g. $\sigma(t) = 0$). Thus, security becomes more meaningful as $\sigma(t) \rightarrow t$. No construction can obtain $\sigma(t) = t$ because by design Eval runs in parallel time t . Ideal security is achieved when $\sigma(t) = t - 1$. This ideal security is in general unrealistic unless, for example, time steps are measured in rounds of queries to an ideal oracle (e.g. random oracle). In practice, if the oracle is instantiated with a concrete program (e.g. a hash function), then differences in hardware/implementation would in general yield small differences in the response time for each query. An almost-perfect VDF would achieve $\sigma(t) = t - o(t)$ sequentiality. Even $\sigma(t) = t - \epsilon t$ sequentiality for small ϵ is sufficient for most applications. Security degrades as $\epsilon \rightarrow 1$. The naive VDF construction combining a hash chain with succinct verifiable computation (i.e. producing a SNARG proof of correctness following the hash chain computation) cannot beat $\epsilon = 1/2$, unless it uses at least $\omega(t)$ parallelism to generate the proof in sublinear time (exceeding the allowable parallelism for VDFs, though see a relaxation to “weak” VDFs below).

Unpredictability and min-entropy. Definition 4 captures an unpredictability property for the output of the VDF, similar to a one-way function. However, similar to random oracles, the output of the VDF on a given input is never indistinguishable from random. It is possible that no depth $\sigma(t)$ circuit can distinguish the output on a randomly sampled challenge from random, but only if the VDF proof is not given to the distinguisher. Efficiently decodable VDFs cannot achieve this stronger property.

For the application to random beacons (e.g. for lotteries), it is only necessary that on a random challenge the output is unpredictable and also has sufficient min-entropy¹ conditioned on previous outputs for different challenges. In fact, σ -sequentiality already implies that min-entropy is $\Omega(\log \lambda)$. Otherwise some fixed output y occurs with probability $1/\text{poly}(\lambda)$ for randomly sampled input x ; the adversary $\mathcal{A}_0$ can compute $O(\text{poly}(\lambda))$ samples of this distribution in the preprocessing to find such a y' with high probability, and then $\mathcal{A}_1$ could output y' as its guess. Moreover, if σ -sequentiality is achieved for t superpolynomial (sub-exponential) in λ , then the preprocessing adversary is allowed $2^{o(\lambda)}$ samples, implying some $o(\lambda)$ min-entropy of the output must be preserved. By itself, σ -sequentiality does not imply $\Omega(\lambda)$ min-entropy. Stronger min-entropy

¹ A randomness extractor can then be applied to the output to map it to a uniform distribution.

preservation can be demonstrated in other ways given additional properties of the VDF, e.g. if it is a permutation or collision-resistant. Under suitable complexity theoretic assumptions (namely the existence of subexponential $2^{o(n)}$ circuit lower bounds) a combination of Nisan-Wigderson type PRGs and extractors can also be used to generate $\text{poly}(\lambda)$ pseudorandom bits from a string with min-entropy $\log \lambda$.

Random “Delay” Oracle. In the random oracle model, *any* unpredictable string (regardless of its min-entropy) can be used to extract an unpredictable λ -bit uniform random string. For the beacon application, a random oracle H would simply be applied to the output of the VDF to generate the beacon value. We can even model this construction as an ideal object itself, a *Random Delay Oracle*, which implements a random function H' and on any given input x it waits for $\sigma(t)$ steps before returning the output $H'(x)$. Demonstrating a construction from a σ -sequential VDF and random oracle H that is provably *indifferentiable* [50] from a Random Delay Oracle is an interesting research question.²

Remark: Removing any single property makes VDF construction easy. We note the existence of well-known outputs if any property is removed:

- If **Verify** is not required to be fast, then simply iterating a one-way function t times yields a trivial solution. Verification is done by re-computing the output, or a set of ℓ intermediate points can be supplied as a proof which can be verified in parallel time $\Theta(t/\ell)$ using ℓ processors, with total verification time remaining $\Theta(t)$.
- If we do not require *uniqueness*, then the construction of Mahmoody et al. [49] using hash functions and depth-robust graphs suffices. This construction was later improved by Cohen and Pietrzak [19]. This construction fails to ensure uniqueness because once an output y is computed it can be easily mangled into many other valid outputs $y' \neq y$, as discussed in Sect. 8.1.
- If we do not require σ -*sequentiality*, many solutions are possible, such as finding the discrete log of a challenge group element with respect to a fixed generator. Note that computing an elliptic curve discrete log can be done in parallel time $o(t)$ using a parallel version of the Pollard rho algorithm [67].

Weaker VDFs. For certain applications it is still interesting to consider a VDF that requires even more than $\text{polylog}(t)$ parallelism in **Eval** to compute the output in parallel time t . For example, in the randomness beacon application only one party is required to compute the VDF and all other parties can simply verify the output. It would not be unreasonable to give this one party a significant amount of parallel computing power and optimized hardware. This would yield

² The difficulty in proving indifferentiability arises because the distinguisher can query the VDF/RO construction and the RO itself separately, therefore the simulator must be able to simulate queries to the random oracle H given only access to the Random Delay Oracle. Indifferentiability doesn't require the simulator to respond in exactly the same time, but it is still required to be *efficient*. This becomes an issue if the delay t is superpolynomial.

a secure beacon as long as no adversary could compute the outputs of *Eval* in faster than t steps given even more parallelism than this party. Moreover, for small values of t it may be practical for anyone to use up to $O(t)$ parallelism (or more). With this in mind, we define a weaker variant of a VDF that allows additional parallelism in *Eval*.

Definition 5. We call a system $V = (\text{Setup}, \text{Eval}, \text{Verify})$ a weak-VDF if it satisfies Definition 1 with the exception that *Eval* is allowed up to $\text{poly}(t, \lambda)$ parallelism.

Note that (p, σ) -sequentiality can only be meaningful for a weak-VDF if *Eval* is allowed strictly less than $p(t)$ parallelism, otherwise the honest computation of *Eval* would require more parallelism than even the adversary is allowed.

4 VDFs from Incrementally Verifiable Computation

VDFs are by definition sequential functions. We therefore require the existence of sequential functions in order to construct any VDF. We begin by defining a sequential function.

Definition 6. ((t, ϵ) -Sequential function). $f : X \rightarrow Y$ is a (t, ϵ) -sequential function if for $\lambda = O(\log(|X|))$, if the following conditions hold.

1. There exists an algorithm that for all $x \in X$ evaluates f in parallel time t using $\text{poly}(\log(t), \lambda)$ processors.
2. For all $\mathcal{A}$ that run in parallel time strictly less than $(1 - \epsilon) \cdot t$ with $\text{poly}(t, \lambda)$ processors:

$$P[y_A = f(x) \mid y_A \stackrel{\mathcal{R}}{\leftarrow} \mathcal{A}(\lambda, x), x \stackrel{\mathcal{R}}{\leftarrow} X] < \text{negl}(\lambda).$$

In addition we consider iterated sequential functions that are an iterative composition of some other function with the same domain and image. The key property of an iterated sequential function cannot be evaluated more efficiently than through iteration of the round function.

Definition 7. (Iterated Sequential Function). Let $g : X \rightarrow X$ be a function which satisfies (t, ϵ) -sequentiality. A function $f : \mathbb{N} \times X \rightarrow X$ defined as $f(k, x) = g^{(k)}(x) = \underbrace{g \circ g \circ \dots \circ g}_{k \text{ times}}$ is called an iterated sequential function (with round function g) if for all $k = 2^{o(\lambda)}$ the function $h : X \rightarrow X$ such that $h(x) = f(k, x)$ is $(k \cdot t, \epsilon)$ -sequential as per Definition 6.

It is widely believed that a chain of a secure hash function (like SHA-256) is an iterated sequential function with $t = O(\lambda)$ and ϵ negligible in λ . The sequentiality of hash chains can be proved in the random oracle model [45, 49]. We will use the functions g explicitly and require it to have an arithmetic circuit representation. Modeling g as an oracle, therefore, does not suffice for our construction.

Another candidate for an iterated sequential function is exponentiation in a finite group of unknown order, where the round function is squaring in the group. The fastest known way to compute this is by repeated squaring which is an iterative sequential computation.

Based on these candidates, we can make the following assumption about the existence of iterated sequential functions:

Assumption 1. *For all $\lambda \in \mathbb{N}$ there exists an ϵ, t with $t = \text{poly}(\lambda)$ and a function $g_\lambda : X \rightarrow X$ s.t. $\log |X| = \lambda$ and X can be sampled in time $\text{poly}(\lambda)$ and g_λ is a (t, ϵ) -sequential function, and the function $f : \mathbb{N} \times X \rightarrow X$ with round function g_λ is an iterated sequential function.*

An iterated sequential function by itself gives us many of the properties needed of a secure VDF construction. It is sequential by definition and the trivial algorithm (iteratively computing g) uses only $\text{poly}(\lambda)$ parallelism. Such a function by itself, however, does not suffice to construct a VDF. The fastest generic verification algorithm simply recomputes the function. While this ensures soundness it does not satisfy the efficient verification requirement of a VDF. The verifier of a VDF needs to be exponentially more efficient than the prover.

SNARGs and SNARKs. A natural idea to improve the verification time is to use verifiable computation. In verifiable computation the prover computes a succinct argument (SNARG) that a computation was done correctly. The argument can be efficiently verified using resources that are independent of the size of the computation. A SNARG is a weaker form of a succinct non-interactive argument of knowledge (SNARK) [35] for membership in an NP language $\mathcal{L}$ with relation R (Definition 8). The additional requirement of a SNARK is that for any algorithm that outputs a valid proof of membership of an instance $x \in \mathcal{L}$ there is also an extractor that “watches” the algorithm and outputs a witness w such that $(x, w) \in R$. In the special case of providing a succinct proof that a (polynomial size) computation F was done correctly, i.e. y is the output of F on x , the NP witness is empty and the NP relation simply consists of pairs $((x, y), \perp)$ such that $F(x) = y$.

Definition 8 (Verifiable Computation/SNARK). *Let $\mathcal{L}$ denote an NP language with relation $R_{\mathcal{L}}$, where $x \in \mathcal{L}$ iff $\exists w R_{\mathcal{L}}(x, w) = 1$. A SNARK system for $R_{\mathcal{L}}$ is a triple of polynomial time algorithms (SNKGen , SNKProve , SNKVerify) that satisfy the following properties:*

Completeness:

$$\forall (x, w) \in R_{\mathcal{L}}, \Pr \left[\text{SNKVerify}(\text{vk}, x, \pi) = 0 \mid \begin{array}{l} (\text{vk}, \text{ek}) \leftarrow \text{SNKGen}(1^\lambda) \\ \pi \leftarrow \text{SNKProve}(\text{ek}, x, w) \end{array} \right] = 0$$

Succinctness: *The length of a proof and complexity of SNKVerify is bounded by $\text{poly}(\lambda, \log(|y| + |w|))$.*

Knowledge extraction:[sub-exponential adversary knowledge extractor] For all adversaries $\mathcal{A}$ running in time $2^{o(\lambda)}$ there exists an extractor $\mathcal{E}_{\mathcal{A}}$ running in time $2^{o(\lambda)}$ such that for all $\lambda \in \mathbb{N}$ and all auxiliary inputs z of size $\text{poly}(\lambda)$:

$$\Pr \left[\begin{array}{c} \text{SNKVerify}(\text{vk}, x, \pi) = 1 \\ R_{\mathcal{L}}(x, w) \neq 1 \end{array} \middle| \begin{array}{c} (\text{vk}, \text{ek}) \leftarrow \text{SNKGen}(1^\lambda) \\ (x, \pi) \leftarrow \mathcal{A}(z, \text{ek}) \\ w \leftarrow \mathcal{E}_{\mathcal{A}}(z, \text{ek}) \end{array} \right] < \text{negl}(\lambda)$$

Impractical VDF from SNARGs. Consider the following construction for a VDF from a t, ϵ -sequential function f . Let $\mathbf{pp} = (\text{ek}, \text{vk}) = \text{SNKGen}(\lambda)$ be the public parameter of a SNARG scheme for proving membership in the language of pairs (x, y) such that $f(x) = y$. On input $x \in X$ the Eval computes $y = f(x)$ and a succinct argument $\pi = \text{SNKProve}(\text{ek}, (x, y), \perp)$. The prover outputs $((x, y), \pi)$. On input $((x, y), \pi)$ the verifier checks $y = f(x)$ by checking $\text{SNKVerify}(\text{vk}, (x, y), \pi) = 1$.

This construction clearly satisfies fast verification. All known SNARK constructions are quasi-linear in the length of the underlying computation f [11]. Assuming the cost for computing a SNARG for a computation of length t is $k \cdot t \log(t)$ then the SNARG VDF construction achieves $\sigma(t) = \frac{(1-\epsilon) \cdot t}{(k+1) \cdot \log(t)}$ sequentiality. This does not even achieve the notion of $(1 - \epsilon')t$ sequentiality for any adversary. This means that the adversary can compute the output of the VDF in a small fraction of the time that it takes the honest prover to convince an honest verifier. If, however, SNKProve is sufficiently parallelizable then it is possible to partially close the gap between the sequentiality of f and the sequentiality of the VDF. The Eval simply executes SNKProve in parallel to reduce the relative total running time compared to the computation of f . SNARK constructions can run in parallel time $\text{polylog}(t)$ on $O(t \cdot \text{polylog}(t))$ processors. This shows that a VDF can theoretically be built from verifiable computation.

The construction has, however, two significant downsides: In practice computing a SNARG is more than 100,000 times more expensive than evaluating the underlying computation [68]. This means that to achieve meaningful sequentiality the SNARG computation would require massive parallelism using hundreds thousands of cores. The required parallelism additionally depends on the time t . Secondly, the construction does not come asymptotically close to the sequentiality induced by the underlying computation f . We, therefore, now give a VDF construction with required parallelism independent of t and σ -sequentiality asymptotically close to $(1 - \epsilon)t$ where ϵ will be defined by the underlying sequential computation.

Incremental Verifiable Computation (IVC). IVC provides a direction for circumventing the problem mentioned above. IVC was first studied by Valiant [66] in the context of computationally sound proofs [51]. Bitansky et al. [14] generalized IVC to distributed computations and to other proof systems such as SNARKs. IVC requires that the underlying computation can be expressed as an iterative sequence of evaluations of the same Turing machine. An iterated sequential function satisfies this requirement.

The basic idea of IVC is that at every incremental step of the computation, a prover can produce a proof that a certain state is indeed the current state of the computation. This proof is updated after every step of the computation to produce a new proof. Importantly, the complexity of each proof in proof size and verification cost is bounded by $\text{poly}(\lambda)$ for any sub-exponential length computation. Additionally the complexity of updating the proof is independent of the total length of the computation.

Towards VDFs from IVC. Consider a VDF construction that runs a sequential computation and after each step uses IVC to update a proof that both this step and the previous proof were correct. Unfortunately, for IVC that requires knowledge extraction we cannot prove soundness of this construction for $t > O(\lambda)$. The problem is that a recursive extraction yields an extractor that is exponential in the recursion depth [14].

The trick around this is to construct a binary tree of proofs of limited depth [14, 66]. The leaf proofs verify computation steps whereas the internal node proofs prove that their children are valid proofs. The verifier only needs to check the root proof against the statement that all computation steps and internal proofs are correct.

We focus on the special case that the function f is an iterated sequential function. The regularity of the iterated function ensures that the statement that the verifier checks is succinct. We impose a strict requirement on our IVC scheme to output both the output of f and a final proof with only an additive constant number of additional steps over evaluating f alone.

We define *tight* IVC for an iterated sequential functions, which captures the required primitive needed for our theoretical VDF. We require that incremental proving is almost overhead free in that the prover can output the proof almost immediately after the computation has finished. The definition is a special case of Valiant's definition [66].

Definition 9 (Tight IVC for iterated sequential functions). Let $f_\lambda : \mathbb{N} \times \mathcal{X} \rightarrow \mathcal{X}$ be an iterated sequential function with round function g_λ having (t, ϵ) -sequentiality. An IVC system for f_λ is a triple of polynomial time algorithms $(\text{IVCGen}, \text{IVCProve}, \text{IVCVerify})$ that satisfy the following properties:

Completeness:

$$\forall x \in \mathcal{X}, \Pr \left[\text{IVCVerify}(\text{vk}, x, y, k, \pi) = \text{Yes} \mid \begin{array}{l} (\text{vk}, \text{ek}) \xleftarrow{\text{R}} \text{IVCGen}(\lambda, f) \\ (y, \pi) \xleftarrow{\text{R}} \text{IVCProve}(\text{ek}, k, x) \end{array} \right] = 1$$

Succinctness: The length of a proof and the complexity of SNKVerify is bounded by $\text{poly}(\lambda, \log(k \cdot t))$.

Soundness: [sub-exponential soundness] For all algorithms $\mathcal{A}$ running in time $2^{o(\lambda)}$:

$$\Pr \left[\begin{array}{l} \text{IVCVerify}(\text{vk}, x, y, k, \pi) = \text{Yes} \\ f(k, x) \neq y \end{array} \mid \begin{array}{l} (\text{vk}, \text{ek}) \xleftarrow{\text{R}} \text{IVCGen}(\lambda, f) \\ (x, y, k, \pi) \xleftarrow{\text{R}} \mathcal{A}(\lambda, \text{vk}, \text{ek}) \end{array} \right] < \text{negl}(\lambda)$$

Tight Incremental Proving: *There exists a k' such that for all $k \geq k'$ and $k = 2^{o(\lambda)}$, $\text{IVCProve}(\text{ek}, k, x)$ runs in parallel time $k \cdot t + O(1)$ using $\text{poly}(\lambda, t)$ -processors.*

Existence of tight IVC. Bitansky et al. [14] showed that any SNARK system such as [59] can be used to construct IVC. Under strong knowledge of exponent assumptions there exists an IVC scheme using a SNARK tree of depth less than λ (Theorem 1 of [14]). In every computation step the prover updates the proof by computing λ new SNARKs each of complexity $\text{poly}(\lambda)$, each verifying another SNARK and one of complexity t which verifies one evaluation of g_λ , the round function of f_λ . Ben Sasson et al. [10] discuss the parallel complexity of the Pinocchio SNARK [59] and show that for a circuit of size m there exists a parallel prover using $O(m \cdot \log(m))$ processors that computes a SNARK in time $O(\log(m))$. Therefore, using these SNARKs we can construct an IVC proof system $(\text{IVCGen}, \text{IVCProve}, \text{IVCVerify})$ where, for sufficiently large t , IVCProve uses $\tilde{O}(\lambda + t)$ parallelism to produce each incremental IVC output in time $\lambda \cdot \log(t + \lambda) \leq t$. If t is not sufficiently large, i.e. $t > \lambda \cdot \log(t + \lambda)$ then we can construct an IVC proof system that creates proofs for k' evaluations of g_λ . The IVC proof system chooses k' such that $t \leq \lambda \cdot \log(k' \cdot t + \lambda)$. Given this the total parallel runtime of IVCProve on k iterations of an (t, ϵ) -sequential function would thus be $k \cdot t + \lambda \cdot \log(k' \cdot t + \lambda) = k \cdot t + O(1)$. This shows that we can construct tight IVC from existing SNARK constructions.

VDF_{IVC} construction. We now construct a VDF from a tight IVC. By Assumption 1 we are given a family $\{f_\lambda\}$, where each $f_\lambda : \mathbb{N} \times X_\lambda \rightarrow X_\lambda$ is defined by $f_\lambda(k, x) = g_\lambda^{(k)}(x)$. Here g_λ is a (s, ϵ) -sequential function on an efficiently sampleable domain of size $O(2^\lambda)$.

Given a tight IVC proof system $(\text{IVCGen}, \text{IVCProve}, \text{IVCVerify})$ for f we can construct a VDF that satisfies $\sigma(t)$ -sequentiality for $\sigma(t) = (1 - \epsilon) \cdot t - O(1)$:

- **Setup** (λ, t) : Let g_λ be a (t, ϵ) -sequential function and f_λ the corresponding iterated sequential function as described in Assumption 1. Run $(\text{ek}, \text{vk}) \xleftarrow{\mathbb{R}} \text{IVCGen}(\lambda, f_\lambda)$. Set k to be the largest integer such that $\text{IVCProve}(\text{ek}, k, x)$ takes time less than t . Output $\mathbf{pp} = ((\text{ek}, k), (\text{vk}))$.
- **Eval** $((\text{ek}, k), x)$: Run $(y, \pi) \xleftarrow{\mathbb{R}} \text{IVCProve}(\text{ek}, k, x)$, output (y, π) .
- **Verify** $(\text{vk}, x, (y, \pi))$: Run and output $\text{IVCVerify}(\text{vk}, x, y, k, \pi)$.

Note that t is fixed in the public parameters. It is, however, also possible to give t directly to **Eval**. VDF_{IVC} is, therefore, *incremental*.

Lemma 1. *VDF_{IVC} satisfies soundness (Definition 3)*

Proof. Assume that an $\text{poly}(t, \lambda)$ algorithm $\mathcal{A}$ outputs (with non-negligible probability in λ) a tuple (x, y, π) on input λ, t , and $\mathbf{pp} \xleftarrow{\mathbb{R}} \text{Setup}(\lambda, t)$ such that $\text{Verify}(\mathbf{pp}, x, y, \pi) = \text{Yes}$ but $f_\lambda(k, x) \neq y$. We can then construct an adversary $\mathcal{A}'$ that violates IVC soundness. Given $(\text{vk}, \text{ek}) \xleftarrow{\mathbb{R}} \text{IVCGen}(\lambda, f_\lambda)$ the adversary $\mathcal{A}'$ runs $\mathcal{A}$ on λ, t , and (vk, ek) . Since (vk, ek) is sampled from the same distribution as $\mathbf{pp} \xleftarrow{\mathbb{R}} \text{Setup}(\lambda, t)$ it follows that, with non-negligible probability in λ ,

$\mathcal{A}'$ outputs (x, y, π) such that $\text{Verify}(\mathbf{pp}, x, y, \pi) = \text{IVCVerify}(\mathbf{vk}, x, y, k, \pi) = \text{Yes}$ and $f_\lambda(k, x) \neq y$, which directly violates the soundness of IVC.

Theorem 1 (VDF_{IVC}). *VDF_{IVC} is a VDF scheme with $\sigma(t) = (1 - \epsilon)t - O(1)$ sequentiality.*

Proof. First note that the VDF_{IVC} algorithms satisfy the definition of the VDF algorithms. IVCProve runs in time $(\frac{t}{s} - 1) \cdot s + s = t$ using $\text{poly}(\lambda, s) = \text{poly}(\lambda)$ processors. IVCVerify runs in total time $\text{poly}(\lambda, \log(t))$. Correctness follows from the correctness of the IVC scheme. Soundness was proved in Lemma 1. The scheme is $\sigma(t)$ -sequential because IVCProve runs in time $k \cdot s + O(1) < t$. If any algorithm that uses $\text{poly}(t, \lambda)$ processors can produce the VDF output in time less than $(1 - \epsilon)t - O(1)$ he can directly break the t, ϵ -sequentiality of f_λ . Since s is independent of t we can conclude that VDF_{IVC} has $\sigma(t) = (1 - \epsilon)t - O(1)$ sequentiality.

5 A Weak VDF Based on Injective Rational Maps

In this section we explore a framework for constructing a weak VDF satisfying $(t^2, o(t))$ -sequentiality based on the existence of degree t injective rational maps that cannot be inverted faster than computing polynomial greatest common denominators (GCDs) of degree t polynomials, which we conjecture cannot be solved in parallel time less than $t - o(t)$ on fewer than t^2 parallel processors. Our candidate map will be a permutation polynomial over a finite field of degree t . The construction built from it is a weak VDF because the Eval will require $O(t)$ parallelism to run in parallel time t .

5.1 Injective Rational Maps

Rational maps on algebraic sets. An *algebraic rational function* on finite vector spaces is a function $F : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$ such that $F = (f_1, \dots, f_m)$ where each $f_i : \mathbb{F}_q^n \rightarrow \mathbb{F}_q$ is a rational function in $\mathbb{F}_q(X_1, \dots, X_n)$, for $i = 1, \dots, m$. An *algebraic set* $\mathcal{Y} \subseteq \mathbb{F}_q^n$ is the complete set of points on which some set S of polynomials simultaneously vanish, i.e. $\mathcal{Y} = \{x \in \mathbb{F}_q^n \mid f(x) = 0 \text{ for all } f \in S\}$ for some $S \subset \mathbb{F}_q[X_1, \dots, X_n]$. An *injective rational map* of algebraic sets $\mathcal{Y} \subseteq \mathbb{F}_q^n$ to $\mathcal{X} \subseteq \mathbb{F}_q^m$ is an algebraic rational function F that is injective on $\mathcal{Y}$, i.e. if $\mathcal{X} := F(\mathcal{Y})$, then for every $\bar{x} \in \mathcal{X}$ there exists a unique $\bar{y} \in \mathcal{Y}$ such that $F(\bar{y}) = \bar{x}$.

Inverting rational maps. Consider the problem of inverting an injective rational map $F = (f_1, \dots, f_m)$ on algebraic sets $\mathcal{Y} \subseteq \mathbb{F}_q^n$ to $\mathcal{X} \subseteq \mathbb{F}_q^m$. Here $\mathcal{Y} \subseteq \mathbb{F}_q^n$ is the set of vanishing points of some set of polynomials S . For $x \in \mathbb{F}_q^m$, a solution to $F(\bar{y}) = \bar{x}$ is a point $\bar{y} \in \mathbb{F}_q^n$ such that all polynomials in S vanish at $\bar{y}$ and $f_i(\bar{y}) = x_i$ for $i = 1, \dots, m$. Furthermore, each $f_i(\bar{y}) = g(\bar{y})/h(\bar{y}) = x_i$ for some polynomials g, h , and hence yields a polynomial constraint $z_i(\bar{y}) := g(\bar{y}) - x_i h(\bar{y}) = 0$. In total we are looking for solutions to $|S| + m$ polynomial constraints on $\bar{y}$.

We illustrate two special cases of injective rational maps that can be inverted by a univariate polynomial GCD computation. In general, inverting injective rational maps on $\mathbb{F}_q^d$ for constant d can be reduced to a univariate polynomial GCD computation using resultants.

- *Rational functions on finite fields.* Consider any injective rational function $F(X) = g(X)/h(X)$, for univariate polynomials h, g , on a finite field $\mathbb{F}_q$. A finite field is actually a special case of an algebraic set over itself; it is the set of roots of the polynomial $X^q - X$. Inverting F on a point $c \in \mathbb{F}_q$ can be done by calculating $GCD(X^q - X, g(X) - c \cdot h(X))$, which outputs $X - s$ for the unique s such that $F(s) = c$.
- *Rational maps on elliptic curves.* An elliptic curve $E(\mathbb{F}_q)$ over $\mathbb{F}_q$ is a 2-dimensional algebraic set of vanishing points in $\mathbb{F}_q^2$ of a bivariate polynomial $E(y, x) = y^2 - x^3 - ax - b$. Inverting an injective rational function F on a point in the image of $F(E(\mathbb{F}_q))$ involves computing the GCD of three bivariate polynomials: E, z_1, z_2 , where z_1 and z_2 come from the two rational function components of F . The resultant $R = Res_y(z_1, z_2)$ is a univariate polynomial in x of degree $deg(z_1) \cdot deg(z_2)$ such that $R(x) = 0$ iff there exists y such that (x, y) is a root of both z_1 and z_2 . Finally, taking the resultant again $R' = Res_y(R, E)$ yields a univariate polynomial such that any root x of R' has a corresponding coordinate y such that (x, y) is a point on E and satisfies constraints z_1 and z_2 . Solving for the unique root of R' reduces to a Euclidean GCD computation as above. Then given x , there are two possible points $(x, y) \in E$, so we can try them both and output the unique point that satisfies all the constraints.

Euclidean algorithm for univariate polynomial GCD. Univariate polynomials over a finite field form a Euclidean domain, and therefore the GCD of two polynomials can be found using the Euclidean algorithm. For two polynomials f and g such that $deg(f) > deg(g) = d$, one first reduces $f \bmod g$ and then computes $GCD(f, g) = GCD(f \bmod g, g)$. In the example $f = X^q - X$, the first step of reducing $X^q \bmod g$ requires $O(\log(q))$ multiplications of degree $O(deg(g))$ polynomials. Starting with X , we run the sequence of repeated squaring operations to get X^q , reducing the intermediate results $\bmod g$ after each squaring operation. Then running the Euclidean algorithm to find $GCD(f \bmod g, g)$ involves $O(d)$ sequential steps where in each step we subtract two $O(d)$ degree polynomials. On a sequential machine this computation takes $O(d^2)$ time, but on $O(d)$ parallel processors this can be computed in parallel time $O(d)$.

NC algorithm for univariate polynomial GCD. There is an algorithm for computing the GCD of two univariate polynomials of degree d in $O(\log^2(d))$ parallel time, but requires $O(d^{3.8})$ parallel processors. This algorithm runs d parallel determinant calculations on submatrices of the Sylvester matrix associated with the two polynomials, each of size $O(d^2)$. Each determinant can be computed in parallel time $O(\log^2(d))$ on $M(d) \in O(d^{2.85})$ parallel processors [25]. The parallel advantage of this method over the euclidean GCD method kicks in after

$O(d^{2.85})$ processors. For any $c \leq d/\log^2(d)$, it is possible to compute the GCD in $O(d/c)$ steps on $c\log^2(d)M(d)$ processors.

Sequentiality of univariate polynomial GCD. The GCD can be calculated in parallel time d using d parallel processors via the Euclidean algorithm. The NC algorithm only beats this bound on strictly greater than $d^{2.85}$ processors, but a hybrid of the two methods can gain an $o(d)$ speedup on only d^2 processors. Specifically, we can run the Euclidean method for $d - d^{2/3}$ steps until we are left with two polynomials of degree $d^{2/3}$, then we can run the NC algorithm using $\log^3(d)M(d^{2/3}) < (d^{2/3})^3 = d^2$ processors to compute the GCD of these polynomials in $O(d^{2/3}/\log(d))$ steps, for a total of $d - \epsilon d^{2/3}$ steps. This improvement can be tightened further, but generally results in $d - o(d)$ steps as long as $M(d) \in \omega(d^2)$.

We pose the following assumption on the parallel complexity of calculating polynomials GCDs on fewer than $O(d^2)$ processors. This assumption would be broken if there is an NC algorithm for computing the determinant of a $n \times n$ matrix on $o(n^2)$ processors, but this would require a significant advance in mathematics on a problem that has been studied for a long time.

Assumption 2. *There is no general algorithm for computing the GCD of two univariate polynomials of degree d over a finite field $\mathbb{F}_q$ (where $q > d^3$) in less than parallel time $d - o(d)$ on $O(d^2)$ parallel processors.*

On the other hand, evaluating a polynomial of degree d can be logarithmic in its degree, provided the polynomial can be expressed as a small arithmetic circuit, e.g. $(ax + b)^d$ can be computed with $O(\log(d))$ field operations.

Abstract weak VDF from an injective rational map. Let $F : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$ be a rational function that is an injective map from $\mathcal{Y}$ to $\mathcal{X} := F(\mathcal{Y})$. We further require that $\mathcal{X}$ is efficiently sampleable and that F can be evaluated efficiently for all $\bar{y} \in \mathcal{Y}$. When using F in a VDF we will require that $|\mathcal{X}| > \lambda t^3$ to prevent brute force attacks, where t and λ are given as input to the **Setup** algorithm.

We will need a family $\mathcal{F} := \{(q, F, \mathcal{X}, \mathcal{Y})\}_{\lambda, t}$ parameterized by λ and t . Given such a family we can construct a weak VDF as follows:

- **Setup**(λ, t): choose a $(q, F, \mathcal{X}, \mathcal{Y}) \in \mathcal{F}$ specified by λ and t , and output $\mathbf{pp} := ((q, F), (q, F))$.
- **Eval**($((q, F), \bar{x})$): for an output $\bar{x} \in \mathcal{X} \subseteq \mathbb{F}_q^m$ compute $\bar{y} \in \mathcal{Y}$ such that $F(\bar{y}) = \bar{x}$; The proof π is empty.
- **Verify**($((q, F), \bar{x}, \bar{y}, \pi)$) outputs Yes if $F(\bar{y}) = \bar{x}$.

The reason we require that F be injective on $\mathcal{Y}$ is so that the solution $\bar{y}$ be unique.

The construction is a weak $(p(t), \sigma(t))$ -VDF for $p(t) = t^2$ and $\sigma(t) = t - o(t)$ assuming that there is no algorithm that can invert $F \in \mathcal{F}$ on a random value in less than parallel time $d - o(d)$ on $O(d^2)$ processors. Note that this is a stronger assumption than 2 as the inversion reduces to a specific GCD computation rather than a general one.

Candidate rational maps. The question, of course, is how to instantiate the function family $\mathcal{F}$ so that the resulting weak VDF system is secure. There are many examples of rational maps on low dimensional algebraic sets among which we can search for candidates. Here we will focus on the special case of efficiently computable permutation polynomials over $\mathbb{F}_q$, and one particular family of permutation polynomials that may be suitable.

5.2 Univariate Permutation Polynomials

The simplest instantiation of the VDF system above is when $n = m = 1$ and $\mathcal{Y} = \mathbb{F}_q$. In this case, the function F is a univariate polynomial $f : \mathbb{F}_q \rightarrow \mathbb{F}_q$. If f implements an injective map on $\mathbb{F}_q$, then it must be a permutation of $\mathbb{F}_q$, which brings us to the study of *univariate permutation polynomials* as VDFs.

The simplest permutation polynomials are the monomials x^e for $e \geq 1$, where $\gcd(e, q - 1) = 1$. These polynomials however, can be easily inverted and do not give a secure VDF. Dickson polynomials [47] $D_{n,\alpha} \in \mathbb{F}_p[x]$ are another well known family of polynomials over $\mathbb{F}_p$ that permute $\mathbb{F}_p$. Dickson polynomials are defined by a recurrence relation and can be evaluated efficiently. Dickson polynomials satisfy $D_{t,\alpha^n}(D_{n,\alpha}(x)) = x$ for all n, t, α where $n \cdot t = 1 \bmod p - 1$, hence they are easy to invert over $\mathbb{F}_p$ and again do not give a secure VDF.

A number of other classes of permutation polynomials have been discovered over the last several decades [38]. We need a class of permutation polynomials over a suitably large field that have a tunable degree, are fast to evaluate (i.e. have $\text{polylog}(d)$ circuit complexity), and cannot be inverted faster than running the parallelized Euclidean algorithm on $O(d)$ processors.

Candidate permutation polynomial. We consider the following polynomial of Guralnick and Muller [37] over $\mathbb{F}_{p^m}$:

$$\frac{(x^s - ax - a) \cdot (x^s - ax + a)^s + ((x^s - ax + a)^2 + 4a^2x)^{(s+1)/2}}{2x^s} \quad (5.1)$$

where $s = p^r$ for odd prime p and a is not a $(s - 1)st$ power in $\mathbb{F}_{p^m}$. This polynomial is a degree s^3 permutation on the field $\mathbb{F}_{p^m}$ for all s, m chosen independently.

Below we discuss why instantiating a VDF with nearly all other examples of permutation polynomials would not be secure and why attacks on these other polynomials do not work against this candidate.

Attacks on other families of permutation polynomials. We list here several other families of permutation polynomials that can be evaluated in $O(\text{polylog}(d))$ time, yet would not yield a secure VDF. We explain why each of these attacks do not work against the candidate polynomial.

1. *Sparse permutation polynomials.* Sparse polynomials have a constant number of terms and therefore can be evaluated in time $O(\log(d))$. There exist families of non-monomial sparse permutation polynomials, e.g. $X^{2^{t+1}+1} + X^3 + X \in$

$\mathbb{F}_{2^{2t+1}}[X]$ [38, Theorem 4.12]. The problem is that the degree of this polynomial is larger than the square root of the field size, which allows for brute force parallel attacks. Unfortunately, all known sparse permutation polynomials have this problem. In our candidate the field size can be made arbitrarily large relative to the degree of the polynomial.

2. *Linear algebraic attacks.* A classic example of a sparse permutation polynomial of tunable degree over an arbitrarily large field, due to Mathieu [33], is the family $x^{p^i} - ax$ over $\mathbb{F}_{p^m}$ where a is not a $p - 1$ st power. Unfortunately, this polynomial is easy to invert because $x \mapsto x^{p^i}$ is a linear operator in characteristic p so the polynomial can be written as a linear equation over an m -dimensional vector space. To prevent linear algebraic attacks the degree of at least one non-linear term in the polynomial cannot be divisible by the field characteristic p . In our candidate there are many such non-linear terms, e.g. of degree $s + 1$ where $s = p^r$.
3. *Exceptional polynomials co-prime to characteristic.* An *exceptional polynomial* is a polynomial $f \in \mathbb{F}_q[X]$ which is a permutation on $\mathbb{F}_{q^m}$ for infinitely many m , which allows us to choose sufficiently large m to avoid brute force attacks. However, all exceptional polynomials over $\mathbb{F}_q$ of degree co-prime to q can be written as the composition of Dickson polynomials and linear polynomials, which are easy to invert [56]. In our candidate, the degree s^3 of the polynomial and field size are both powers of p , and are therefore not co-prime.

Additional application: a new family of one-way permutations. We note that a sparse permutation polynomial of sufficiently high degree over a sufficiently large finite field may be a good candidate for a one-way permutation. This may give a secure one-way permutation over a domain of smaller size than what is possible by other methods.

5.3 Comparison to Square Roots mod p

A classic approach to designing a sequentially slow verifiable function, dating back to Dwork and Naor [31], is computing modular square roots. Given a challenge $x \in \mathbb{Z}_p^*$, computing $y = x^{\frac{p+1}{4}} \pmod{p}$ can be efficiently verified by checking that $y^2 = x \pmod{p}$ (for $p \equiv 3 \pmod{4}$). There is no known way to compute this exponentiation in faster than $\log(p)$ sequential field multiplications.

This is a special case of inverting a rational function over a finite field, namely the polynomial $f(y) = y^2$, although this function is not injective and therefore cannot be calculated with GCDs. An injective rational function with nearly the same characteristics is the permutation $f(y) = y^3$. Since the inverse of 3 mod $p - 1$ will be $O(\log p)$ bits, this requires $O(\log p)$ squaring operations to invert. Viewed another way, this degree 3 polynomial can be inverted on a point c by computing the GCD($y^p - y, y^2 - c$), where the first step requires reducing $y^p - y \pmod{y^3 - c}$, involving $O(\log p)$ repeated squarings and reductions mod $y^3 - c$.

While this approach appears to offer a delay parameter of $t = \log(p)$, as t grows asymptotically the evaluator can use $O(t)$ parallel processors to gain a factor t parallel speedup in field multiplications, thus completing the challenge in parallel time equivalent to one squaring operation on a sequential machine. Therefore, there is asymptotically no difference in the parallel time complexity of the evaluation and the total time complexity of the verification, which is why this does not even meet our definition of a weak VDF. Our approach of using higher degree injective rational maps gives a strict (asymptotic) improvement on the modular square/cubes approach, and to the best of our knowledge is the first concrete algebraic candidate to achieve an exponential gap between parallel evaluation complexity and total verification complexity.

6 Practical Improvements on VDFs from IVC

In this section we propose a practical boost to constructing VDFs from IVC (Sect. 4). In an IVC construction the prover constructs a SNARK which verifies a SNARK. Ben-Sasson et al. [11] showed an efficient construction for IVC using “cycles of Elliptic curves”. This construction builds on the pairing-based SNARK [59]. This SNARK system operates on arithmetic circuits defined over a finite field $\mathbb{F}_p$. The proof output consists of elements of an elliptic curve group $E/\mathbb{F}_q$ of prime order p (defined over a field $\mathbb{F}_q$). The SNARK verification circuit, which computes a pairing, is therefore an arithmetic circuit over $\mathbb{F}_q$. Since $q \neq p$, the prover cannot construct a new SNARK that directly operates on the verification circuit, as the SNARK operates on circuits defined over $\mathbb{F}_p$. Ben-Sasson et al. propose using two SNARK systems where the curve order of one is equal to the base field of the other, and vice versa. This requires finding a pair of pairing-friendly elliptic curves E_1, E_2 (defined over two different base fields $\mathbb{F}_1$ and $\mathbb{F}_2$) with the property that the order of each curve is equal to the size of the base field of the other.

The main practical consideration in VDF_{IVC} is that the evaluator needs to be able to update the incremental SNARK proofs at the same rate as computing the underlying sequential function, and without requiring a ridiculous amount of parallelism to do so. Our proposed improvements are based on two ideas:

1. In current SNARK/IVC constructions (including [11, 59]) the prover complexity is proportional to the multiplicative arithmetic complexity of the underlying statement over the field $\mathbb{F}_p$ used in the SNARK ($p \approx 2^{128}$). Therefore, as an optimization, we can use a “SNARK friendly” hash function (or permutation) as the iterated sequential function such that the verification of each iteration has a lower multiplicative arithmetic complexity over $\mathbb{F}_p$.
2. We can use the Eval of a weak VDF as the iterated sequential function, and compute a SNARK over the Verify circuit applied to each incremental output instead of the Eval circuit. This should increase the number of sequential steps required to evaluate the iterated sequential function relative to the number of multiplication gates over which the SNARK is computed.

An improvement of type (1) alone could be achieved by simply using a cipher or hash function that has better multiplicative complexity over the SNARK field $\mathbb{F}_q$ than AES or SHA256 (e.g., see MiMC [5], which has 1.6% complexity of AES). We will explain how using square roots in $\mathbb{F}_q$ or a suitable permutation polynomial over $\mathbb{F}_q$ (from Sect. 5) as the iterated function achieve improvements of both types (1) and (2).

6.1 Iterated Square Roots in $\mathbb{F}_q$

Sloth. A recent construction called Sloth [46] proposed a secure way to chain a series of square root computations in $\mathbb{Z}_p$ interleaved with a simple permutation³ such that the chain must be evaluated sequentially, i.e. is an iterated sequential function (Definition 7). More specifically, Sloth defines two permutations on $\mathbb{F}_p$: a permutation ρ such that $\rho(x)^2 = \pm x$, and a permutation σ such that $\sigma(x) = x \pm 1$ depending on the parity of x . The parity of x is defined as the integer parity of the unique $\hat{x} \in \{0, \dots, p-1\}$ such that $\hat{x} = x \pmod p$. Then Sloth iterates the permutation $\tau = \rho \circ \sigma$.

The verification of each step in the chain requires a single multiplication over $\mathbb{Z}_p$ compared to the $O(\log(p))$ multiplications required for evaluation. Increasing the size of p amplifies this gap, however it also introduces an opportunity for parallelizing multiplication in $\mathbb{Z}_p$ for up to $O(\log(p))$ speedup.

Using Sloth inside VDF_{IVC} would only achieve a practical benefit if $p = q$ for the SNARK field $\mathbb{F}_q$, as otherwise implementing multiplication in $\mathbb{Z}_p$ in an arithmetic circuit over $\mathbb{F}_q$ would have $O(\log^2(p))$ complexity. On modern architectures, multiplication of integers modulo a 256-bit prime is near optimal on a single core, whereas multi-core parallelized algorithms only offer speed-ups for larger primes [8]. Computing a single modular square root for a 256-bit prime takes approximately 570 cycles on an Intel Core i7 [46], while computing SHA256 for 256-bit outputs takes approximately 864 cycles⁴. Therefore, to achieve the same wall-clock time-delay as an iterated SHA256 chain, only twice as many iterations of modular square roots are needed.

The best known arithmetic circuit implementation of SHA256 has 27,904 multiplication gates [9]. In stark contrast, the arithmetic circuit over $\mathbb{F}_p$ for verifying a modular square root is a single multiplication gate. Verifying the permutation σ is more complex as it requires a parity check, but this requires at most $O(\log(p))$ complexity.

Sloth++ extension. Replacing SHA256 with Sloth as the iterated function in VDF_{IVC} already gives a significant improvement, as detailed above. Here we suggest yet a further optimization, which we call Sloth++. The main arithmetic complexity of verifying a step of Sloth comes from the fact that the permutation σ is not naturally arithmetic over $\mathbb{F}_p$, which was important for preventing attacks

³ If square roots are iterated on a value x without an interleaved permutation then there is a shortcut to the iterated computation that first computes $v = (\frac{p+1}{4})^\ell \pmod p$ and then the single exponentiation x^v .

⁴ <http://www.ouah.org/ogay/sha2/>.

that factor $\tau^\ell(x)$ as a polynomial over $\mathbb{F}_p$. Our idea here is to compute square roots over a degree 2 extension field $\mathbb{F}_{p^2}$ interleaved with a permutation that is arithmetic over $\mathbb{F}_p$ but not over $\mathbb{F}_{p^2}$.

In any degree r extension field $\mathbb{F}_{p^r}$ of $\mathbb{F}_p$ for a prime $p \equiv 3 \pmod{4}$ a square root of an element $x \in \mathbb{F}_{p^r}$ can be found by computing $x^{(p^r+1)/4}$. This is computed in $O(r \log(p))$ repeated squaring operations in $\mathbb{F}_{p^r}$. Verifying a square root requires a single multiplication over $\mathbb{F}_{p^r}$. Elements of $\mathbb{F}_{p^r}$ can be represented as length r vectors over $\mathbb{F}_p$, and each multiplication reduces to $O(r^2)$ arithmetic operations over $\mathbb{F}_p$. For $r = 2$ the verification *multiplicative* complexity over $\mathbb{F}_p$ is exactly 4 gates.

In Sloth++ we define the permutation ρ exactly as in Sloth, yet over $\mathbb{F}_{p^2}$. Then we define a simple non-arithmetic permutation σ on $\mathbb{F}_{p^2}$ that swaps the coordinates of elements in their vector representation over $\mathbb{F}_p$ and adds a constant, i.e. maps the element (x, y) to $(y + c_1, x + c_2)$. The arithmetic circuit over $\mathbb{F}_p$ representing the swap is trivial: it simply swaps the values on the input wires. The overall multiplicative complexity of verifying an iteration of Sloth++ is only 4 gates over $\mathbb{F}_p$. Multiplication can be parallelized for a factor 2 speedup, so 4 gates must be verified roughly every 1700 parallel-time evaluation cycles. Thus, for parameters that achieve the same wall-clock delay, the SNARK verification complexity of Sloth++ is a 14,000 fold improvement over that of a SHA256 chain.

Cube roots. The underlying permutation in both Sloth and Sloth++ can be replaced by cube roots over $\mathbb{F}_q$ when $\gcd(3, q - 1) = 1$. In this case the slow function is computing $\rho(x) = x^v$ where $3v \equiv 1 \pmod{q - 1}$. The output can be verified as $\rho(x)^3 = x$.

6.2 Iterated Permutation Polynomials

Similar to Sloth+, we can use our candidate permutation polynomial (Eq. 5.1) over $\mathbb{F}_q$ as the iterated function in VDF_{IVC} . Recall that $\mathbb{F}_q$ is an extension field chosen independently from the degree of the polynomial. We would choose $q \approx 2^{256}$ and use the same $\mathbb{F}_q$ as the field used for the SNARK system. For each $O(d)$ sequential provers steps required to invert the polynomial on a point, the SNARK only needs to verify the evaluation of the polynomial on the inverse, which has multiplicative complexity $O(\log(d))$ over $\mathbb{F}_q$. Concretely, for each 10^5 parallel-time evaluation cycles a SNARK needs to verify approximately 16 gates. This is yet another factor 15 improvement over Sloth+. The catch is that the evaluator must use 10^5 parallelism⁵ to optimize the polynomial GCD computation. We must also assume that an adversary cannot feasibly amass more than 10^{14} parallel processors to implement the NC parallelized algorithm for polynomial GCD.

⁵ This is reasonable if the evaluator has an NVIDIA Titan V GPU, which can compute up to 10^{14} pipelined arithmetic operations per second (<https://www.nvidia.com/en-us/titan/titan-v/>).

From a theory standpoint, using permutation polynomials inside VDF_{IVC} reduces it to a weak VDF because the degree of the polynomial must be super-polynomial in λ to prevent an adversary from implementing the NC algorithm on $\text{poly}(\lambda)$ processors, and therefore the honest evaluator is also required to use super-polynomial parallelism. However, the combination does yield a better weak VDF, and from a practical standpoint appears quite promising for many applications.

7 Towards VDFs from Exponentiation in a Finite Group

The sequential nature of large exponentiation in a finite group may appear to be a good source for secure VDF systems. This problem has been used extensively in the past for time-based problems such as time-lock puzzles [64], benchmarking [21], timed commitments [16], and client puzzles [31, 46]. Very recently, Pietrzak [61] showed how to use this problem to construct a VDF that requires a trusted setup. The trusted setup can be eliminated by instead choosing a sufficiently large random number N so that N has two large prime factors with high probability. However, the large size of N provides the adversary with more opportunity for parallelizing the arithmetic. It also increases the verifier's running time. Alternatively, one can use the class group of an imaginary quadratic order [20], which is an efficient group of unknown order with a public setup [48].

7.1 Exponentiation-Based VDFs with Bounded Pre-computation

Here we suggest a simple exponentiation-based approach to constructing VDFs whose security would rely on the assumption that the adversary cannot run a long pre-computation between the time that the public parameters $\mathbf{pp}$ are made public and the time when the VDF needs to be evaluated. Therefore, in terms of security this construction is subsumed by the more recent solution of Pietrzak [61], however it yields much shorter proofs. We use the following notation to describe the VDF:

- let $L = \{\ell_1, \ell_2, \dots, \ell_t\}$ be the first t odd primes, namely $\ell_1 = 3$, $\ell_2 = 5$, etc. Here t is the provided delay parameter.
- let P be the product of the primes in L , namely $P := \ell_1 \cdot \ell_2 \cdots \ell_t$. This P is a large integer with about $t \log t$ bits.

With this notation, the trusted setup procedure works as follows: construct an RSA modulus N , say 4096 bits long, where the prime factors are strong primes. The trusted setup algorithm knows the factorization of N , but no one else will. Let $\mathbb{G} := (\mathbb{Z}/N\mathbb{Z})^*$. We will also need a random hash function $H : \mathbb{Z} \rightarrow \mathbb{G}$. Next, for a given preprocessing security parameter B , say $B = 2^{30}$, do:

- for $i = 1, \dots, B$: compute $h_i \leftarrow H(i) \in \mathbb{G}$ and then compute $g_i := h_i^{1/P} \in \mathbb{G}$.

– output

$$\mathbf{ek} := (\mathbb{G}, H, g_1, \dots, g_B) \quad \text{and} \quad \mathbf{vk} := (\mathbb{G}, H).$$

Note that the verifier's public parameters are short, but the evaluators parameters are not.

Solving a challenge x : Algorithm $\mathbf{Eval}(\mathbf{pp}_{\text{eval}}, x)$ takes as input the public parameters $\mathbf{pp}_{\text{eval}}$ and a challenge $x \in \mathcal{X}$.

- using a random hash function, map the challenge x to a random subset $L_x \subseteq L$ of size λ , and a random subset S_x of λ values in $\{1, \dots, B\}$.
- Let P_x be the product of all the primes in L_x , and let g be $g := \prod_{i \in S_x} g_i \in \mathbb{G}$.
- the challenge solution y is simply $y \leftarrow g^{P/P_x} \in \mathbb{G}$, which takes $O(t \log t)$ multiplications in $\mathbb{G}$.

Verifying a solution y : Algorithm $\mathbf{Verify}(\mathbf{pp}_{\text{verify}}, x, y)$ works as follows:

- Compute P_x and S_x as in algorithm $\mathbf{Eval}(\mathbf{pp}_{\text{eval}}, x)$.
- let h be $h := \prod_{i \in S_x} H(i) \in \mathbb{G}$.
- output yes if and only if $y^{P_x} = h$ in $\mathbb{G}$.

Note that exactly one $y \in \mathbb{G}$ will be accepted as a solution for a challenge x . Verification takes only $\tilde{O}(\lambda)$ group operations.

Security. The scheme does not satisfy the definition of a secure VDF, but may still be useful for some of the applications described in Sect. 2. In particular, the system is not secure against an adversary who can run a large pre-computation once the parameters $\mathbf{pp}$ are known. There are several pre-computation attacks possible that require tB group operations in $\mathbb{G}$. Here we describe one such instructive attack. It uses space $O(sB)$, for some $s > 0$, and gives a factor of s speed up for evaluating the VDF.

Consider the following pre-computation, for a given parameter s , say $s = 100$. Let $b = \lfloor P^{1/s} \rfloor$, then the adversary computes and stores a table of size sB :

$$\text{for all } i = 1, \dots, B: \quad g_i^b, g_i^{(b^2)}, \dots, g_i^{(b^s)} \in \mathbb{G}. \quad (7.1)$$

Computing these values is comparable to solving B challenges. Once computed, to evaluate the VDF at input x , the adversary uses the precomputed table to quickly compute

$$g^b, g^{(b^2)}, \dots, g^{(b^s)} \in \mathbb{G}.$$

Now, to compute g^{P/P_x} , it can write P/P_x in base b as:
 $P/P_x = \alpha_0 + \alpha_1 b + \alpha_2 b^2 + \dots + \alpha_s b^s$ so that

$$g^{P/P_x} = g^{\alpha_0} \cdot (g^b)^{\alpha_1} \cdot (g^{(b^2)})^{\alpha_2} \dots (g^{(b^s)})^{\alpha_s}.$$

This expression can be evaluated in parallel and gives a parallel adversary a factor of s speed-up over a sequential solver, which violates the sequentiality property of the VDF.

To mount this attack, the adversary must compute the entire table (7.1) for all $g_1, \dots, g_B$, otherwise it can only gain a factor of two speed-up with negligible probability in λ . Hence, the scheme is secure for only B challenges, after which new public parameters need to be generated. This may be sufficient for some applications of a VDF.

8 Related Work

Taking a broad perspective, VDFs can be viewed as an example of *moderately hard* cryptographic functions. Moderately hard functions are those whose difficulty to compute is somewhere in between ‘easy’ (designed to be as efficient as possible) and ‘hard’ (designed to be so difficult as to be intractable). The use of moderately hard cryptographic functions dates back at least to the use of a deliberately slow DES variant for password hashing in early UNIX systems [55]. Dwork and Naor [31] coined the term *moderately hard* in a classic paper proposing client puzzles or “pricing functions” for the purpose of preventing spam. Juels and Brainard proposed the related notion of a *client puzzle*, in which a TCP server creates a puzzle which must be solved before a client can open a connection [42]. Both concepts have been studied for a variety of applications, including TLS handshake requests [7, 29], node creation in peer-to-peer networks [30], creation of digital currency [27, 57, 63] or censorship resistance [18]. For interactive client puzzles, the most common construction is as follows: the server chooses a random ℓ -bit value x and sends to the client $H(x)$ and $x[\ell - \log_2 t - 1]$. The client must send back the complete value of x . That is, the server sends the client $H(x)$ plus all of the bits of x except the final $\log_2 t + 1$ bits, which the client must recover via brute force.

8.1 Inherently Sequential Puzzles

The simple interactive client puzzle described above is embarrassingly parallel and can be solved in constant time given t processors. In contrast, the very first construction of a client puzzle proposed by Dwork and Naor involved computing modular square roots and is believed to be inherently sequential (although they did not discuss this as a potential advantage).

The first interest in designing puzzles that require an inherently sequential solving algorithm appears to come for the application of hardware benchmarking. Cai et al. [21, 22] proposed the use of inherently sequential puzzles to verify claimed hardware performance as follows: a customer creates an inherently-sequential puzzle and sends it to a hardware vendor, who then solves it and returns the solution (which the customer can easily verify) as quickly as possible. Note that this work predated the definition of client puzzles. Their original construction was based on exponentiation modulo an RSA number N , for which the customer has created N and therefore knows $\varphi(N)$. They later proposed solutions based on a number of other computational problems not typically used

in cryptography, including Gaussian elimination, fast Fourier transforms, and matrix multiplication.

Time-lock puzzles. Rivest, Shamir, and Wagner [64] constructed a time-lock encryption scheme, also based on the hardness of RSA factoring and the conjectured sequentiality of repeated exponentiation in a group of unknown order. The encryption key K is derived as $K = x^{2^t} \in \mathbb{Z}_N$ for an RSA modulus N and a published starting value x . The encrypting party, knowing $\varphi(N)$, can reduce the exponent $e = 2^t \bmod \varphi(N)$ to quickly derive $K = x^e \bmod N$. The key K can be publicly recovered slowly by 2^t iterated squarings. Boneh and Naor [16] showed that the puzzle creator can publish additional information enabling an efficient and sound proof that K is correct. In the only alternate construction we are aware of, Bitansky et al. [15] show how to construct time-lock puzzles from randomized encodings assuming any inherently-sequential functions exist.

Time-lock puzzles are similar to VDFs in that they involve computing an inherently sequential function. However, time-lock puzzles are defined in a private-key setting where the verifier uses its private key to prepare each puzzle (and possibly a verification proof for the eventual answer). In contrast to VDFs, this trusted setup must be performed per-puzzle and each puzzle takes no unpredictable input.

Proofs of sequential work. Mahmoody et al. [49] proposed publicly verifiable proofs of sequential work (PoSW) which enable proving to any challenger that a given amount of sequential work was performed on a specific challenge. As noted, time-lock puzzles are a type of PoSW, but they are not publicly verifiable. VDFs can be seen as a special case of publicly verifiable proofs of sequential work with the additional guarantee of a unique output (hence the use of the term “function” versus “proof”).

Mahmoody et al.’s construction uses a sequential hash function H (modeled as a random oracle) and depth robust directed-acyclic graph G . Their puzzle involves computing a *labeling* of G using H salted by the challenge c . The label on each node is derived as a hash of all the labels on its parent nodes. The labels are committed to in a Merkle tree and the proof involves opening a randomly sampled fraction. Very briefly, the security of this construction is related to graph pebbling games (where a pebble can be placed on a node only if all its parents already have pebbles) and the fact that depth robust graphs remain sequentially hard to pebble even if a constant fraction of the nodes are removed (in this case corresponding to places where the adversary cheats). Mahmoody et al. proved security unconditionally in the random oracle model. Depth robust graphs and parallel pebbling hardness are used similarly to construct memory hard functions [40] and proofs of space [32]. Cohen and Pietrzak [19] constructed a similar PoSW using a simpler non-depth-robust graph based on a Merkle tree.

PoSWs based on graph labeling don’t naturally provide a VDF because removing any single edge in the graph will change the output of the proof, yet is unlikely to be detected by random challenges.

Sequentially hard functions. The most popular solution for a slow function which can be viewed as a proto-VDF, dating to Dwork and Naor [31], is computing modular square roots. Given a challenge $x \in \mathbb{Z}_p^*$, computing $y = x^{\frac{p+1}{4}} \pmod{p}$ can be efficiently verified by checking that $y^2 = x \pmod{p}$ (for $p \equiv 3 \pmod{4}$). There is no known algorithm for computing modular exponentiation which is sublinear in the exponent. However, the difficulty of puzzles is fixed to $t = \log p$ as the exponent can be reduced modulo $p - 1$ before computation, requiring the use of a very large prime p to produce a difficult puzzle.

This puzzle has been considered before for similar applications as our VDFs, in particular randomness beacons [39, 46]. Lenstra and Wesolowski [46] proposed creating a more difficult puzzle for a small p by chaining a series of such puzzles together (interleaved with a simple permutation) in a construction called Sloth. We proposed a simple improvement of this puzzle in Sect. 6. Recall that this does not meet our asymptotic definition of a VDF because it does not offer (asymptotically) efficient verification, however we used it as an important building block to construct a more practical VDF based on IVC. Asymptotically, Sloth is comparable to a hash chain of length t with t checkpoints provided as a proof, which also provides $O(\text{polylog}(t))$ -time verification (with t processors) and a solution of size $\Theta(t \cdot \lambda)$.

9 Conclusions

Given their large number of interesting applications, we hope this work stimulates new practical uses for VDFs and continued study of theoretical constructions. We still lack a theoretically optimal VDF, consisting of a simple inherently sequential function requiring low parallelism to compute but yet being very fast (e.g. logarithmic) to invert. These requirements motivate the search for new problems which have not traditionally been used in cryptography. Ideally, we want a VDF that is also post-quantum secure.

Acknowledgments. We thank Micheal Zieve for his help with permutation polynomials. We thank the CRYPTO reviewers for their helpful comments. This work was supported by NSF, a grant from ONR, the Simons Foundation, and a Google faculty fellowship.

References

1. RANDAO: A DAO working as RNG of Ethereum. Technical report (2016)
2. Filecoin: A decentralized storage network. Protocol Labs (2017). <https://filecoin.io/filecoin.pdf>
3. Proof of replication. Protocol Labs (2017). <https://filecoin.io/proof-of-replication.pdf>
4. Threshold relay. Dfinity (2017). <https://dfinity.org/pdfs/viewer.html?file=../library/threshold-relay-blockchain-stanford.pdf>

5. Albrecht, M., Grassi, L., Rechberger, C., Roy, A., Tiessen, T.: MiMC: efficient encryption and cryptographic hashing with minimal multiplicative complexity. In: Cheon, J.H., Takagi, T. (eds.) ASIACRYPT 2016. LNCS, vol. 10031, pp. 191–219. Springer, Heidelberg (2016). https://doi.org/10.1007/978-3-662-53887-6_7
6. Armknecht, F., Barman, L., Bohli, J.-M., Karame, G.O.: Mirror: enabling proofs of data replication and retrievability in the cloud. In: USENIX Security Symposium, pp. 1051–1068 (2016)
7. Aura, T., Nikander, P., Leiwo, J.: DOS-resistant authentication with client puzzles. In: Christianson, B., Malcolm, J.A., Crispo, B., Roe, M. (eds.) Security Protocols 2000. LNCS, vol. 2133, pp. 170–177. Springer, Heidelberg (2001). https://doi.org/10.1007/3-540-44810-1_22
8. Baktir, S., Savas, E.: Highly-parallel montgomery multiplication for multi-core general-purpose microprocessors. In: Gelenbe, E., Lent, R. (eds.) Computer and Information Sciences III, pp. 467–476. Springer, London (2013). https://doi.org/10.1007/978-1-4471-4594-3_48
9. Ben-Sasson, E., et al. Zerocash: decentralized anonymous payments from Bitcoin. In: IEEE Symposium on Security and Privacy (2014)
10. Ben-Sasson, E., Chiesa, A., Genkin, D., Tromer, E., Virza, M.: SNARKs for C: verifying program executions succinctly and in zero knowledge. In: Canetti, R., Garay, J.A. (eds.) CRYPTO 2013. LNCS, vol. 8043, pp. 90–108. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-40084-1_6
11. Ben-Sasson, E., Chiesa, A., Tromer, E., Virza, M.: Scalable zero knowledge via cycles of elliptic curves. *Algorithmica* **79**, 1102–1160 (2014)
12. Bentov, I., Gabizon, A., Zuckerman, D.: Bitcoin beacon. arXiv preprint [arXiv:1605.04559](https://arxiv.org/abs/1605.04559) (2016)
13. Bentov, I., Pass, R., Shi, E.: Snow white: provably secure proofs of stake. IACR Cryptology ePrint Archive, 2016 (2016)
14. Bitansky, N., Canetti, R., Chiesa, A., Tromer, E.: Recursive composition and bootstrapping for SNARKs and proof-carrying data. In: Proceedings of the Forty-Fifth Annual ACM Symposium on Theory of Computing, pp. 111–120. ACM (2013)
15. Bitansky, N., Goldwasser, S., Jain, A., Paneth, O., Vaikuntanathan, V., Waters, B.: Time-lock puzzles from randomized encodings. In: ACM Conference on Innovations in Theoretical Computer Science (2016)
16. Boneh, D., Naor, M.: Timed commitments. In: Bellare, M. (ed.) CRYPTO 2000. LNCS, vol. 1880, pp. 236–254. Springer, Heidelberg (2000). https://doi.org/10.1007/3-540-44598-6_15
17. Bonneau, J., Clark, J., Goldfeder, S.: On bitcoin as a public randomness source (2015). <https://eprint.iacr.org/2015/1015.pdf>
18. Bonneau, J., Xu, R.: Scrambling for lightweight censorship resistance. In: Christianson, B., Crispo, B., Malcolm, J., Stajano, F. (eds.) Security Protocols 2011. LNCS, vol. 7114, pp. 296–302. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-25867-1_28
19. Cohen, B., Pietrzak, K.: Simple proofs of sequential work. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018. LNCS, vol. 10821, pp. 451–467. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-78375-8_15
20. Buchmann, J., Williams, H.C.: A key-exchange system based on imaginary quadratic fields. *J. Cryptol.* **1**(2), 107–118 (1988)
21. Cai, J., Lipton, R.J., Sedgewick, R., Yao, A.C.: Towards uncheatable benchmarks. In: Structure in Complexity Theory (1993)
22. Cai, J.-Y., Nerurkar, A., Wu, M.-Y.: The design of uncheatable benchmarks using complexity theory (1997)

23. Cascudo, I., David, B.: Scrape: scalable randomness attested by public entities. Cryptology ePrint Archive, Report 2017/216 (2017). <http://eprint.iacr.org/2017/216>
24. Clark, J., Hengartner, U.: On the use of financial data as a random beacon. In: Usenix EVT/WOTE (2010)
25. Codenottia, B., Datta, B.N., Datta, K., Leoncini, M.: Parallel algorithms for certain matrix computations. *Theor. Comput. Sci.* **180**, 287–308 (1997)
26. Cohen, B.: Proofs of space and time. In: Blockchain Protocol Analysis and Security Engineering (2017). <https://cyber.stanford.edu/sites/default/files/bramcohen.pdf>
27. Dai, W.: B-money. Consulted **1**, 2012 (1998)
28. David, B., Gazi, P., Kiayias, A., Russell, A.: Ouroboros Praos: an adaptively-secure, semi-synchronous proof-of-stake blockchain. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018. LNCS, vol. 10821, pp. 66–98. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-78375-8_3
29. Dean, D., Stubblefield, A.: Using client puzzles to protect TLS. In: USENIX Security Symposium, vol. 42 (2001)
30. Douceur, J.R.: The Sybil attack. In: Druschel, P., Kaashoek, F., Rowstron, A. (eds.) IPTPS 2002. LNCS, vol. 2429, pp. 251–260. Springer, Heidelberg (2002). https://doi.org/10.1007/3-540-45748-8_24
31. Dwork, C., Naor, M.: Pricing via processing or combatting junk mail. In: Brickell, E.F. (ed.) CRYPTO 1992. LNCS, vol. 740, pp. 139–147. Springer, Heidelberg (1993). https://doi.org/10.1007/3-540-48071-4_10
32. Dziembowski, S., Faust, S., Kolmogorov, V., Pietrzak, K.: Proofs of space. In: Gennaro, R., Robshaw, M. (eds.) CRYPTO 2015. LNCS, vol. 9216, pp. 585–605. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-48000-7_29
33. Mathieu, É.: Mémoire sur l'étude des fonctions de plusieurs quantités sur la manière de les former et sur les substitutions qui les laissent invariables. *J. Math. Pures Appl.* **6**(2), 241–323 (1861)
34. Garay, J., Kiayias, A., Leonardos, N.: The Bitcoin backbone protocol: analysis and applications. Cryptology ePrint Archive # 2014/765 (2014)
35. Gennaro, R., Gentry, C., Parno, B., Raykova, M.: Quadratic span programs and succinct NIZKs without PCPs. In: Johansson, T., Nguyen, P.Q. (eds.) EUROCRYPT 2013. LNCS, vol. 7881, pp. 626–645. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-38348-9_37
36. Goldschlag, D.M., Stubblebine, S.G.: Publicly verifiable lotteries: applications of delaying functions. In: Hirschfeld, R. (ed.) FC 1998. LNCS, vol. 1465, pp. 214–226. Springer, Heidelberg (1998). <https://doi.org/10.1007/BFb0055485>
37. Guralnick, R.M., Müller, P.: Exceptional polynomials of affine type. *J. Algebra* **194**(2), 429–454 (1997)
38. Hou, X.-d.: Permutation polynomials over finite fields a survey of recent advances. *Finite Fields Appl.* **32**, 82–119 (2015)
39. Jerschow, Y.I., Mauve, M.: Non-parallelizable and non-interactive client puzzles from modular square roots. In: Availability, Reliability and Security (ARES) (2011)
40. Alwen, J., Blocki, J., Pietrzak, K.: Depth-robust graphs and their cumulative memory complexity. In: Coron, J.-S., Nielsen, J.B. (eds.) EUROCRYPT 2017. LNCS, vol. 10212, pp. 3–32. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-56617-7_1
41. Juels, A., Kaliski Jr., B.S.: PORs: proofs of retrievability for large files. In: Proceedings of the 14th ACM Conference on Computer and Communications Security, pp. 584–597. ACM (2007)

42. Jules, A., Brainard, J.: Client-puzzles: a cryptographic defense against connection depletion. In: Proceedings of Network and Distributed System Security Symposium (NDSS 1999), pp. 151–165 (1999)
43. Kiayias, A., Russell, A., David, B., Oliynykov, R.: Ouroboros: a provably secure proof-of-stake blockchain protocol. In: Katz, J., Shacham, H. (eds.) CRYPTO 2017. LNCS, vol. 10401, pp. 357–388. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-63688-7_12
44. King, S., Nadal, S.: Peercoin—secure & sustainable cryptocoin, August 2012. <https://peercoin.net/whitepaper>
45. Kogan, D., Manohar, N., Boneh, D.: T/key: second-factor authentication from secure hash chains. In: ACM Conference on Computer and Communications Security (2017)
46. Lenstra, A.K., Wesolowski, B.: A random zoo: sloth, unicorn, and trx. IACR Cryptology ePrint Archive, 2015 (2015)
47. Lidl, R., Mullen, G.L., Turnwald, G.: Dickson Polynomials, vol. 65. Chapman & Hall/CRC, Boca Raton (1993)
48. Lipmaa, H.: Secure accumulators from euclidean rings without trusted setup. In: Bao, F., Samarati, P., Zhou, J. (eds.) ACNS 2012. LNCS, vol. 7341, pp. 224–240. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-31284-7_14
49. Mahmoody, M., Moran, T., Vadhan, S.: Publicly verifiable proofs of sequential work. In: Proceedings of the 4th Conference on Innovations in Theoretical Computer Science. ACM (2013)
50. Maurer, U., Renner, R., Holenstein, C.: Indifferentiability, impossibility results on reductions, and applications to the random oracle methodology. In: Naor, M. (ed.) TCC 2004. LNCS, vol. 2951, pp. 21–39. Springer, Heidelberg (2004). https://doi.org/10.1007/978-3-540-24638-1_2
51. Micali, S.: CS proofs. In: 1994 Proceedings of the 35th Annual Symposium on Foundations of Computer Science, pp. 436–453. IEEE (1994)
52. Micali, S.: Algorand: the efficient and democratic ledger. arXiv preprint [arXiv:1607.01341](https://arxiv.org/abs/1607.01341) (2016)
53. Miller, A., Juels, A., Shi, E., Parno, B., Katz, J.: Permacoin: repurposing bitcoin work for data preservation. In: 2014 IEEE Symposium on Security and Privacy (SP), pp. 475–490. IEEE (2014)
54. Moran, T., Naor, M., Segev, G.: An optimally fair coin toss. In: Reingold, O. (ed.) TCC 2009. LNCS, vol. 5444, pp. 1–18. Springer, Heidelberg (2009). https://doi.org/10.1007/978-3-642-00457-5_1
55. Morris, R., Thompson, K.: Password security: a case history. Commun. ACM **22**(11), 594–597 (1979)
56. Müller, P.: A weil-bound free proof of Schur’s conjecture. Finite Fields Appl. **3**(1), 25–32 (1997)
57. Nakamoto, S.: Bitcoin: a peer-to-peer electronic cash system (2008)
58. Park, S., Pietrzak, K., Kwon, A., Alwen, J., Fuchsbauer, G., Gai, P.: SpaceMint: a cryptocurrency based on proofs of space. Cryptology ePrint Archive, Report 2015/528 (2015). <http://eprint.iacr.org/2015/528>
59. Parno, B., Howell, J., Gentry, C., Raykova, M.: Pinocchio: nearly practical verifiable computation. In: IEEE Security and Privacy (2013)
60. Pierrot, C., Wesolowski, B.: Malleability of the blockchains entropy. Cryptogr. Commun. **10**, 211–233 (2016)
61. Pietrzak, K.: Unique proofs of sequential work from time-lock puzzles (2018). Manuscript

62. Rabin, M.O.: Transaction protection by beacons. *J. Comput. Syst. Sci.* **27**, 256–267 (1983)
63. Rivest, R.L., Shamir, A.: PayWord and MicroMint: two simple micropayment schemes. In: Lomas, M. (ed.) *Security Protocols 1996*. LNCS, vol. 1189, pp. 69–87. Springer, Heidelberg (1997). https://doi.org/10.1007/3-540-62494-5_6
64. Rivest, R.L., Shamir, A., Wagner, D.A.: Time-lock puzzles and timed-release crypto (1996)
65. Syta, E., et al.: Scalable bias-resistant distributed randomness. In: *2017 IEEE Symposium on Security and Privacy (SP)*, pp. 444–460. IEEE (2017)
66. Valiant, P.: Incrementally verifiable computation or proofs of knowledge imply time/space efficiency. In: Canetti, R. (ed.) *TCC 2008*. LNCS, vol. 4948, pp. 1–18. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-78524-8_1
67. Van Oorschot, P.C., Wiener, M.J.: Parallel collision search with application to hash functions and discrete logarithms. In: *ACM Conference on Computer and Communications Security* (1994)
68. Wahby, R.S., Setty, S.T., Ren, Z., Blumberg, A.J., Walfish, M.: Efficient RAM and control flow in verifiable outsourced computation. In: *NDSS* (2015)